

SOAP in Web Services



☐ soap in webservices

SOAP in Web Services

SOAP stands for **Simple Object Access Protocol**.

It is a **protocol** used for communication between applications over a network, mainly in **Web Services**.

SOAP allows programs written in different languages and running on different platforms to exchange data in a standard format.

Basic Definition

SOAP is an **XML-based messaging protocol** used to send and receive information between client and server in web services.

It commonly works over:

- HTTP
- HTTPS
- SMTP
- TCP

Why SOAP Was Created

Different systems use different technologies:

- Java
- .NET
- Python
- PHP
- Linux
- Windows

SOAP provides a **common communication format** so all systems can interact.

Example:

A banking server in Java can communicate with a .NET client using SOAP messages.

Important Features of SOAP

1. XML Based

SOAP messages are written entirely in XML.

Example structure:

XML

```
<soap:Envelope>
  <soap:Body>
    <getStudent>
      <id>101</id>
    </getStudent>
  </soap:Body>
</soap:Envelope>
```

2. Platform Independent

SOAP works on any operating system and programming language.

3. Language Independent

A Java application can communicate with a Python application.

4. Standard Protocol

SOAP follows strict standards defined by W3C.

Because of this, it is highly reliable.

5. Supports Security

SOAP supports:

- Encryption
- Authentication
- Digital signatures

Using standards like:

- WS-Security

This is why SOAP is heavily used in:

- Banking
 - Healthcare
 - Enterprise systems
-

SOAP Architecture

SOAP message mainly contains:

1. Envelope

Defines the start and end of the SOAP message.

It tells:

"This is a SOAP message."

2. Header (Optional)

Contains additional information such as:

- Authentication
 - Security
 - Transactions
-

3. Body

Contains the actual request or response data.

4. Fault (Optional)

Used for error handling.

Example:

XML

```
<soap:Fault>  
  <faultcode>500</faultcode>  
  <faultstring>Server Error</faultstring>  
</soap:Fault>
```

SOAP Communication Flow

Step-by-step

Step 1: Client sends SOAP Request

The client sends an XML request to the server.

Step 2: Server Processes Request

The server reads the XML and performs the required operation.

Step 3: Server sends SOAP Response

The server returns the result in XML format.

Example of SOAP Request

```
XML
POST /studentService HTTP/1.1
Host: example.com
Content-Type: text/xml
```

```
<soap:Envelope>
  <soap:Body>
    <getStudent>
      <id>101</id>
    </getStudent>
  </soap:Body>
</soap:Envelope>
```

Example of SOAP Response

```
XML
<soap:Envelope>
  <soap:Body>
    <getStudentResponse>
      <name>Sushant</name>
      <course>CSE</course>
    </getStudentResponse>
  </soap:Body>
</soap:Envelope>
```

What is WSDL?

SOAP services usually use:

WSDL (Web Services Description Language)

WSDL is an XML document that describes:

- Service name
- Available methods

- Parameters
- Return types
- Service location

It acts like a contract between client and server.

Advantages of SOAP

1. High Security

Supports enterprise-level security.

2. Reliable Communication

Supports ACID transactions and guaranteed delivery.

3. Standardized Protocol

Strict standards improve interoperability.

4. Works Across Platforms

Can communicate between heterogeneous systems.

Disadvantages of SOAP

1. Complex

SOAP messages are large and difficult compared to REST.

2. Slower

XML processing increases overhead.

3. More Bandwidth

SOAP messages are verbose.

SOAP vs REST

Feature	SOAP	REST
Type	Protocol	Architectural style
Data Format	XML only	JSON, XML, Text
Speed	Slower	Faster
Security	Very high	Moderate
Complexity	Complex	Simple
Usage	Enterprise apps	Web/mobile apps

Real-World Uses of SOAP

SOAP is commonly used in:

- Banking systems
- Payment gateways
- Airline reservation systems
- Enterprise applications
- Government systems

Viva Questions and Answers

Q1. What is SOAP?

SOAP is an XML-based protocol used for communication between applications in web services.

Q2. Why is SOAP called platform independent?

Because it can work on any operating system and programming language.

Q3. Which language is used in SOAP messages?

XML.

Q4. What are the main parts of a SOAP message?

- Envelope
 - Header
 - Body
 - Fault
-

Q5. What is WSDL?

WSDL is an XML document that describes a SOAP web service.

Q6. Why is SOAP more secure than REST?

SOAP supports WS-Security, encryption, authentication, and digital signatures.

Q7. What is the role of the SOAP Body?

It contains the actual request or response data.

Q8. What is SOAP Fault?

SOAP Fault is used for handling errors in SOAP messages.

Counter Questions Examiner May Ask

“Why does SOAP use XML only?”

Because XML was chosen as a universal standard that is platform and language independent.

“Why is SOAP slower?”

Because XML messages are large and require more parsing and processing.

“Why do banks still use SOAP?”

Because SOAP provides:

- High security
- Reliability
- Transaction support
- Standardization

which are critical for financial systems.

“Can SOAP work without HTTP?”

Yes. SOAP can work over:

- SMTP
 - TCP
 - JMS
 - HTTP
-

Simple Analogy

Think of SOAP like a **registered courier service**:

- Strict rules
- Secure delivery
- Tracking
- Reliable

REST is more like a normal fast delivery service:

- Faster
- Simpler
- Lightweight

SOAP prioritizes reliability and security over speed.



☐ explain struts

Struts Framework

What is Struts?

Apache Struts is an **open-source Java framework** used for developing **web applications**.

It is mainly based on the **MVC architecture** (Model-View-Controller).

Struts helps developers create:

- Large-scale enterprise web applications
- Maintainable applications
- Structured applications

It simplifies handling:

- Request processing
 - Form handling
 - Validation
 - Navigation between pages
-

Full Form of Struts

There is no official full form, but it represents a framework that provides a strong structure ("struts") for Java web applications.

Why Struts Was Introduced

Before Struts, developers used:

- Servlets
- JSP pages

directly for web applications.

Problems with this approach:

- Business logic mixed with presentation
- Difficult maintenance
- Poor scalability
- Code duplication

Struts solved this using the MVC design pattern.

MVC Architecture in Struts

1. Model

Contains:

- Business logic
- Database operations
- JavaBeans / Services

Example:

Java
StudentService.java

2. View

User interface shown to users.

Usually:

- JSP pages
- HTML

Example:

HTML
login.jsp

3. Controller

Controls the flow between View and Model.

In Struts:

- Front Controller = **ActionServlet**

It receives all requests from users.

Architecture Flow of Struts

Step-by-Step Working

Step 1: User Sends Request

Example:

/login.do

Step 2: ActionServlet Receives Request

ActionServlet is the central controller.

It checks configuration in:

XML
struts-config.xml

Step 3: ActionForm Stores Data

Form data entered by the user is stored in:

Java
ActionForm

Step 4: Validation Happens

Struts validates the form data.

Example:

- Empty username
- Invalid password

Step 5: Action Class Executes Business Logic

Example:

Java
LoginAction.java

This class interacts with:

- Database
- Services
- Business logic

Step 6: Result Returned to JSP

Success or failure page is shown to the user.

Example:

- success.jsp
- error.jsp

Main Components of Struts

1. ActionServlet

Acts as the controller.

Responsibilities:

- Receives requests
 - Manages workflow
 - Calls appropriate Action class
-

2. Action Class

Contains business logic.

Example:

Java
`public class LoginAction extends Action`

3. ActionForm

Stores user input data temporarily.

Example:

Java
`username`
`password`

4. struts-config.xml

Configuration file of Struts.

Contains:

- Action mappings
- Form beans
- Navigation rules

Example:

XML
`<action path="/login"`
`type="LoginAction"`
`input="/login.jsp">`

5. JSP

Used for displaying UI.

Example of Struts Login Flow

User opens:

`login.jsp`

User enters:

- Username
- Password

↓

Request goes to:

ActionServlet

↓

Mapped to:

LoginAction

↓

Checks database

↓

Returns:

- success.jsp
- OR
- error.jsp

Features of Struts

1. MVC Support

Separates:

- UI
- Business logic
- Control flow

2. Reusable Components

Code can be reused easily.

3. Centralized Configuration

All mappings handled in:

XML
`struts-config.xml`

4. Form Validation

Built-in validation support.

5. Tag Libraries

Provides custom JSP tags.

Example:

JSP
`<html:form>`

Advantages of Struts

1. Better Code Organization

Uses MVC architecture.

2. Easy Maintenance

Changes in one layer do not affect others much.

3. Reusability

Action classes and forms can be reused.

4. Enterprise Level Support

Suitable for large applications.

Disadvantages of Struts

1. Complex Configuration

Large XML configuration files.

2. Slower Development

Compared to modern frameworks.

3. Older Technology

Now less popular compared to:

- Spring Framework
 - Spring Boot
-

Struts 1 vs Struts 2

Feature	Struts 1	Struts 2
Controller	ActionServlet	FilterDispatcher
Form Handling	ActionForm	POJO
Validation	Limited	Better
Flexibility	Less	More
Integration	Difficult	Easier

Real-World Usage

Struts was widely used in:

- Banking systems
- Insurance portals
- Enterprise applications

Many legacy enterprise systems still use it.

Viva Questions and Answers

Q1. What is Struts?

Struts is an open-source Java MVC framework used for developing enterprise web applications.

Q2. Which architecture does Struts follow?

MVC architecture.

Q3. What is the controller in Struts?

ActionServlet.

Q4. What is ActionForm?

A JavaBean used to store form data submitted by the user.

Q5. What is the purpose of struts-config.xml?

It stores configuration details like action mappings and navigation rules.

Q6. What is an Action class?

A class that contains business logic and processes requests.

Q7. Why is Struts called a framework?

Because it provides predefined structure and components for web application development.

Q8. Which technologies are used with Struts?

- Java
 - JSP
 - Servlets
 - XML
-

Counter Questions Examiner May Ask

“Why do we need MVC?”

MVC separates:

- UI
- Logic
- Request handling

making applications easier to manage and maintain.

“Why is ActionServlet called Front Controller?”

Because all requests first go through it before reaching other components.

“Why is Struts less popular now?”

Modern frameworks like:

- Spring Boot
- Hibernate

provide:

- Less configuration
 - Faster development
 - Better annotations support
-

“Can Struts work without JSP?”

Yes, but JSP is commonly used as the View layer.

Simple Analogy

Think of Struts like a restaurant system:

- Customer = User
- Waiter = Controller (**ActionServlet**)
- Chef = Business Logic (**Action**)
- Menu Form = **ActionForm**
- Kitchen Rules = **struts-config.xml**

- Food Served = JSP Response

The waiter controls everything between customer and kitchen, just like Struts manages requests and responses.